ADB Misconfiguration Detection Lab

ADB Misconfiguration Detection Lab

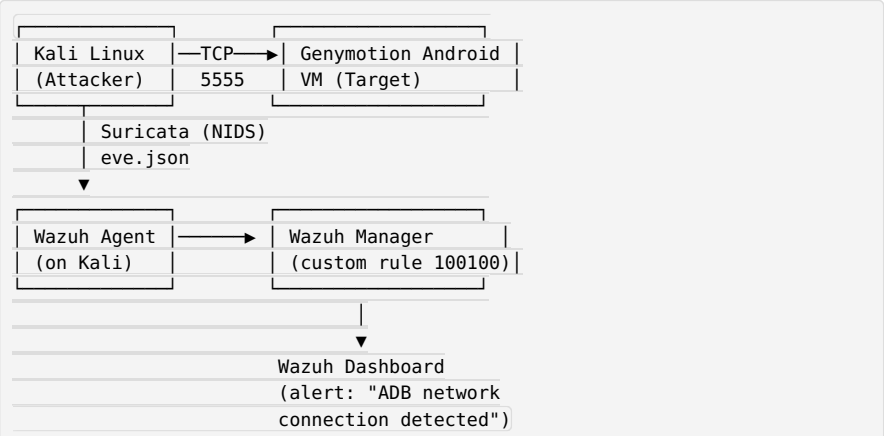
Simulating and detecting a real-world Android Debug Bridge (ADB) network exposure, using a Genymotion virtual device as the target, Kali Linux as the attacker, and Suricata + Wazuh as the detection stack.

This lab is a follow-up to my SOC Detection Lab (Wazuh + Kali + Ubuntu endpoint), extending the same detection-engineering approach to a mobile attack surface.

Objective

Demonstrate, end to end, how an exposed ADB service on a mobile device can be discovered and exploited by an attacker on the same network, and how that activity can be detected and alerted on through a SIEM pipeline.

Architecture

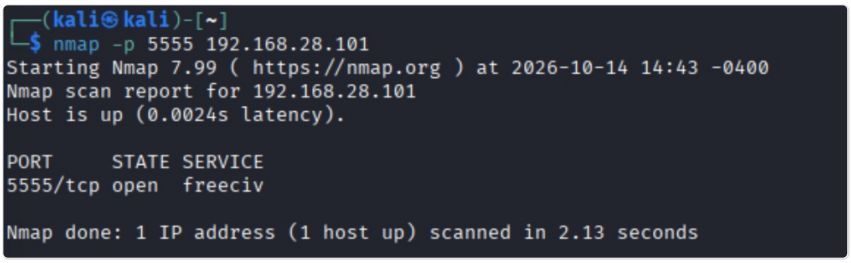


Tools Used

Tool	Purpose
Genymotion	Android virtual device (simulated target)
Kali Linux	Attack platform
nmap	Port discovery / recon
ADB (Android Debug Bridge)	Exploitation of the exposed debug interface
Suricata	Network intrusion detection, custom rule
Wazuh	SIEM — log aggregation, custom detection rule, alerting

Attack Walkthrough

1. **Target setup** — Genymotion Android VM (`192.168.28.101`), network-accessible ADB (TCP 5555), simulating a device left in a debuggable state.
2. **Recon** — `nmap -p 5555 192.168.28.101` from Kali confirms the port is open.



3. **Exploitation** — `adb connect 192.168.28.101:5555` establishes a connection with no authentication required.



4. **Impact demonstration** — `adb shell screenrecord` remotely records the device screen; `adb pull` retrieves the recording to the attacker machine as evidence of full device control. See `demo.mp4` .

Detection Walkthrough

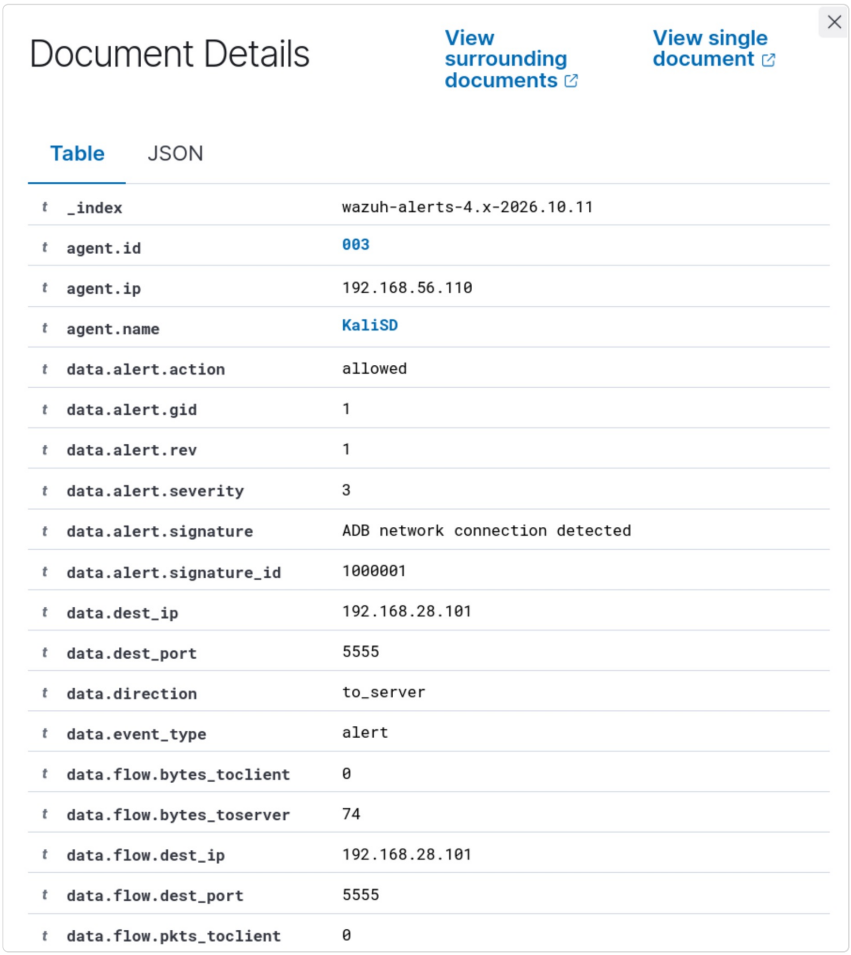
1. **Suricata rule** (`/etc/suricata/rules/local.rules` on Kali):

```
alert tcp any any -> 192.168.28.101 5555 (msg:"ADB network connection
detected"; sid:1000001; rev:1;)
```

2. Suricata logs the match to `/var/log/suricata/eve.json` .
3. A Wazuh agent on the Kali host monitors `eve.json` as a JSON log source.
4. A custom Wazuh rule on the manager (`local_rules.xml`) matches the Suricata alert signature and raises a level-10 alert:

```
        <rule id="100100" level="10">
          <decoded_as>json</decoded_as>
          <field name="data.alert.signature">ADB network connection
detected</field>
          <description>Suricata: ADB network connection detected on Kali -
possible remote access attempt</description>
          <group>suricata,adb_detection,</group>
        </rule>
```

5. The alert surfaces in the Wazuh dashboard in real time, carrying the full Suricata payload (signature, source/dest IP and port):



Document Details

View surrounding documents

View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.10.11
t agent.id	003
t agent.ip	192.168.56.110
t agent.name	KaliSD
t data.alert.action	allowed
t data.alert.gid	1
t data.alert.rev	1
t data.alert.severity	3
t data.alert.signature	ADB network connection detected
t data.alert.signature_id	1000001
t data.dest_ip	192.168.28.101
t data.dest_port	5555
t data.direction	to_server
t data.event_type	alert
t data.flow.bytes_toclient	0
t data.flow.bytes_toserver	74
t data.flow.dest_ip	192.168.28.101
t data.flow.dest_port	5555
t data.flow.pkts_toclient	0

Remediation

- Disable network ADB (`adb tcpip`) when not actively debugging; revert to USB-only (`adb usb`).
- Leave ADB in USB-only mode by default on production/real devices.
- Firewall TCP 5555 at the network level if remote debugging must stay enabled.

Design Notes / Troubleshooting

A few real-world findings worth calling out from building this lab:

- **Windows Firewall only logs host-local traffic.** An earlier approach tried detecting the attack via Windows Firewall logging, but VM-to-VM traffic on a VirtualBox host-only network never touches the Windows host's own network stack — it's invisible to host-based firewall logging. This was the reason for pivoting to a network-based IDS (Suricata) running directly on the attacking host instead.
- **Wazuh doesn't auto-alert on arbitrary JSON.** Ingesting a JSON log source doesn't produce dashboard alerts by itself — a custom rule matching the specific JSON fields is required.

Repo Contents

- `nmap-scan.png` — port discovery evidence

- `adb-connect.png` — successful connection
- `demo.mp4` — remote screen recording (attack impact)
- `wazuh-alert.png` — SIEM alert detail, showing the Suricata signature and the destination IP/port it matched
- `local.rules` — Suricata detection rule
- `local_rules.xml` — Wazuh custom detection rule

Steven Doyley GitHub · LinkedIn